

Parallel Computing with GPUs

Report on CUDA and OpenMP Implementations of
Count Gliders, Histogram, and Emboss Algorithms

Nour Elashry

University of Sheffield

Advanced Computer Science MSc

May 20, 2026

Contents

Table of Contents	i
1 Introduction	1
2 Implementation	2
2.1 OpenMP	2
2.1.1 Count Glider	2
2.1.2 Histogram	3
2.1.3 Emboss	5
2.2 CUDA	7
2.2.1 Count Gliders	7
2.2.2 Histogram	9
2.2.3 Emboss	11
3 Benchmarking	15
3.1 Count Gliders	15
3.2 Histogram	16
3.3 Emboss	17
3.4 Count Gliders: Extra	18
3.5 Emboss: Extra	19
3.6 NSight Compute Profiling Results	19
3.6.1 Count Gliders	20
3.6.2 Histogram	20
3.6.3 Emboss	21
4 Conclusion	22

Chapter 1

Introduction

This report investigates the implementation and optimisation of parallel algorithms using two distinct parallel computing frameworks, namely OpenMP for multicore CPU execution, and CUDA for GPU-based (i.e. many-core) execution. The objective is to evaluate how different hardware architectures influence algorithmic performance, and how parallelisation strategies must be configured, designed, and adapted to effectively exploit these architectures. Particular emphasis is placed on performance analysis via benchmarking, which makes for a grounded comparison of scalability and efficiency between implementations.

Chapter 2

Implementation

2.1 OpenMP

2.1.1 Count Glider

Description of Approach

The OpenMP implementation parallelises traversal of the input grid while preserving the original glider-matching logic from the CPU version. Since each 3×3 window can be checked independently, the outer traversal loop is suitable for data parallelism.

Parallelism was introduced using:

```
#pragma omp parallel
{
#pragma omp for reduction(+:count) schedule(static)
```

The `for` directive distributes iterations of the outer x -loop between threads, allowing multiple image regions to be processed simultaneously. The original nested matching logic over `reflect`, `rotate`, `gx`, and `gy` was retained unchanged.

The implementation still uses the `cell_index` pointer array from the CPU version to simplify 2D indexing while preserving the flattened memory layout of the original image.

Justification of Approach

The traversal stage dominates execution time because every valid 3×3 position in the image must be tested against multiple glider transformations. Each iteration is independent since the result of one window does not affect any neighbouring window. This makes the traversal naturally suitable for loop-level parallelism using OpenMP's `parallel for` model.

The outer loop was selected for parallelisation because it exposes a large number of independent iterations while avoiding modification of the inner matching logic. Parallelising the smaller inner

loops would introduce substantially higher thread-management overhead relative to the amount of work performed per iteration.

The clause `reduction(+:count)` was required because multiple threads may detect gliders concurrently. Reduction avoids race conditions by giving each thread a private partial counter before combining the final sums once execution completes.

Static scheduling was selected because the workload per iteration is approximately uniform. Each thread performs the same sequence of reflection, rotation, and comparison operations for each candidate window. Since work distribution is predictable, static scheduling avoids the runtime overhead associated with dynamic scheduling.

Limitation of Approach

The implementation parallelises traversal but does not reduce the computational cost of the matching stage itself. Each thread still performs repeated reflection and rotation checks across both glider groups, producing a large amount of branch-heavy work inside the nested loops.

The use of `goto` for early exits reduces unnecessary comparisons after mismatches are detected, but it also introduces irregular control flow. Different iterations may therefore execute different numbers of operations depending on how early mismatches occur.

Scaling is also limited by CPU resource sharing. Although iterations are independent, threads still compete for cache capacity and memory bandwidth, so performance does not scale linearly with thread count.

Comparison with CPU Implementation

The OpenMP implementation preserves the same traversal and matching procedure as the CPU version. Both implementations test every valid 3×3 window against the two glider groups and all corresponding reflections and rotations.

The difference lies in how traversal work is distributed. The CPU version processes windows serially, while the OpenMP version divides outer-loop iterations between multiple threads using `#pragma omp for`.

The matching logic itself remains unchanged. As a result, the OpenMP implementation improves throughput through concurrent execution rather than algorithmic redesign.

2.1.2 Histogram

Description of Approach

The OpenMP implementation parallelises traversal of the input array while avoiding concurrent writes to shared histogram bins. Since multiple input values may map to the same bin, directly updating the shared `output` array would introduce race conditions.

To avoid this, each thread accumulates results into a private local histogram stored inside `all_hists`:

```
int* local_hist =  
    all_hists + (size_t)tid * HISTOGRAM_LEN;
```

The input traversal is parallelised using `#pragma omp for`. Each thread processes a subset of the input array, computes the corresponding bin through integer division, and increments its own local histogram:

```
size_t bin = (size_t)(numbers[i] / bin_width);  
++local_hist[bin];
```

Once parallel traversal completes, all local histograms are merged into the final output histogram through a serial accumulation stage.

Justification of Approach

Histogram construction creates frequent write collisions because many input values may map to the same output bin. Using atomic operations on the shared `output` array would therefore introduce substantial synchronisation overhead, particularly when the input distribution repeatedly targets the same bins.

Thread-local histograms remove this contention during traversal. Since each thread writes exclusively to its own histogram, the main accumulation stage proceeds without synchronisation.

This approach also preserves the natural data parallelism of histogram construction. Each input value can be processed independently, making the traversal stage well suited to OpenMP loop parallelism.

Static work distribution was sufficient because each iteration performs the same small amount of work: one division and one histogram update. As a result, workload imbalance between threads is minimal.

Limitation of Approach

The implementation increases memory usage by allocating one histogram per thread:

$$\text{num_threads} \times \text{HISTOGRAM_LEN}$$

rather than a single shared histogram.

The final merge stage is also serial in the current implementation. Although substantially smaller than the traversal stage, it still introduces additional overhead which increases with both thread count and histogram size.

Performance also depends on cache behaviour during merging. Since multiple local histograms are accumulated into the same output array, repeated accesses to neighbouring bins may increase cache pressure.

Comparison with CPU Implementation

The CPU implementation updates the shared histogram directly during traversal:

```
++output[numbers[i] / bin_width];
```

Since execution is serial, no synchronisation is required.

The OpenMP implementation preserves the same binning logic but replaces direct shared updates with thread-local accumulation. Each thread constructs its own histogram during traversal before all local histograms are merged into the final output array.

The OpenMP version therefore introduces additional memory usage and a merge stage, but avoids the synchronisation overhead associated with concurrent updates to shared bins.

2.1.3 Emboss

Description of Approach

The OpenMP implementation parallelises output pixel generation. Each output pixel is independent because it is computed only from the corresponding 3×3 neighbourhood in the input image, not from neighbouring output values.

The two traversal loops over the output image were parallelised using:

```
#pragma omp for collapse(2)
```

The `collapse(2)` clause combines the y and x loops into one iteration space. Each iteration computes one output pixel by reading the required RGB values, converting them to greyscale, applying the emboss kernel, adding the normalising offset of 128, clamping the result to $[0, 255]$, and writing it to `output[out_offset]`.

The emboss mask is unchanged from the CPU version:

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Justification of Approach

Emboss is a stencil-style computation: each output value gathers a fixed local neighbourhood from the input image. This makes the output traversal directly parallelisable, since separate threads can

compute separate output pixels without communicating.

The `collapse(2)` clause was used because the work is naturally two-dimensional. Distributing only the outer loop would give OpenMP fewer chunks of work, especially when the image height is small relative to the number of available threads. Collapsing both loops exposes `OUT_WIDTH × OUT_HEIGHT` independent iterations.

No reduction, atomic operation, or critical section is required. Each iteration writes to a unique output index:

```
out_offset = y * OUT_WIDTH + x;
```

so two threads never update the same output location.

Limitation of Approach

The OpenMP version parallelises traversal but does not reduce the work required per output pixel. Each pixel still performs nine RGB loads, nine greyscale conversions, nine kernel multiplications, accumulation, normalisation, and clamping.

Neighbouring output pixels also reuse many of the same input pixels, but this implementation does not explicitly cache or share those values between threads. Repeated reads and repeated greyscale conversions remain part of the cost.

Performance is therefore limited by both arithmetic work and memory behaviour. The access pattern is local, but each output pixel still gathers several neighbouring RGB values from the input image.

Comparison with CPU Implementation

The CPU and OpenMP versions use the same emboss kernel, greyscale conversion, normalisation, clamping, and output dimensions. The computed result is therefore unchanged.

The difference is the traversal strategy. The CPU version computes output pixels serially, while the OpenMP version distributes the collapsed two-dimensional iteration space across multiple threads.

Unlike the histogram implementation, the emboss implementation does not need thread-local storage or a merge step. Parallelism is safe because each output pixel has exactly one writer.

2.2 CUDA

2.2.1 Count Gliders

Description of Approach

The CUDA implementation maps the glider search onto a two-dimensional grid of GPU threads, where each thread evaluates one possible 3×3 starting coordinate. Since only $(w-2)(h-2)$ regions are valid, the kernel is launched over the reduced search space:

```
dim3 grid(  
    (width - 2 + CG_BLOCK_W - 1) / CG_BLOCK_W,  
    (height - 2 + CG_BLOCK_H - 1) / CG_BLOCK_H  
);
```

A 16×16 block size gives 256 threads per block, corresponding to eight full warps. Each block declares shared memory for the image tile and warp-level partial counts:

```
__shared__ unsigned char tile[CG_TILE_H][CG_TILE_W];  
__shared__ unsigned int warpCounts[CG_WARPS_PER_BLOCK];
```

The tile includes halo cells because neighbouring threads inspect overlapping 3×3 regions. Threads cooperatively load the tile from global memory, after which `__syncthreads()` ensures the shared-memory values are fully initialised before matching begins. The two glider templates are stored in constant memory because they are small, read-only, and repeatedly accessed throughout matching. Each valid thread compares its local region against both glider templates under all reflection and rotation combinations. The helper function `areSame()` performs comparisons directly from shared memory and exits immediately after mismatches are detected.

After matching, the kernel performs warp-level reduction:

```
unsigned int mask =  
    __ballot_sync(0xffffffff, found);  
  
unsigned int matches =  
    __popc(mask);
```

One thread per warp stores the partial result in shared memory. Thread zero then combines the warp totals and performs one global `atomicAdd()` per block.

The host function handles device allocation, memory transfers, kernel launch, result copy-back, and memory release.

```

PS C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master> & "C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.6\compute-sanitizer\compute-sanitizer.exe" --tool memcheck --leak-check full "C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master\x64\Release\assignment.exe" CUDA CG "C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master\cg_16_in.png" --bench
===== COMPUTE-SANITIZER
Using input file: C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master\cg_16_in.png
Image has 4 channels, converting to 1.
Input has dimensions: 25x25
100/100
Count Gliders Result: Pass
CPU: 16
CUDA: 16
CUDA average execution timing from 100 runs
Using GPU: NVIDIA GeForce MX570 A
Time: 0.610ms
===== LEAK SUMMARY: 0 bytes leaked in 0 allocations
===== ERROR SUMMARY: 0 errors
PS C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master> |

```

Figure 2.1: Checking for Leaks in the CUDA Count Gliders Algorithm

The implementation was also tested using NVIDIA Compute Sanitizer with leak checking enabled, shown in Figure 2.1. The output reported 0 bytes leaked and 0 detected errors, confirming correct device-memory management throughout execution.

Justification of Approach

The implementation exposes parallelism across the full search space by assigning one CUDA thread to each valid 3×3 starting coordinate. Since each region can be checked independently, the traversal naturally follows a data-parallel execution model.

Shared-memory tiling was used because neighbouring threads inspect overlapping 3×3 regions. Caching a block-sized region together with halo cells reduces redundant global-memory accesses and improves locality between neighbouring threads.

The two glider templates were stored in constant memory because they are small, read-only, and repeatedly accessed throughout matching.

The counting stage was designed to reduce global atomic contention. Instead of performing one `atomicAdd()` per matching thread, the implementation first performs reduction at warp level using `__ballot_sync()` and `__popc()`, then combines warp totals at block level before issuing one global atomic update per block.

Limitations of Approach

The main limitation is warp divergence. Threads in the same warp may leave `areSame()` after different numbers of comparisons because some 3×3 regions fail earlier than others. Some threads may also stop after finding a match while others continue checking more rotations and reflections. Since warps execute together, some threads must wait while others keep working.

The implementation also remains comparison-heavy. One coordinate may still require testing two base patterns, several rotations and reflections, and up to nine cells per transformed pattern. Shared-memory tiling reduces repeated global-memory access, but it does not reduce the comparison work itself.

The reduction stage greatly decreases global atomic contention by combining matches at warp and block level before the final update. However, each block still performs one global `atomicAdd()` to the same counter, so blocks may still wait when several updates occur simultaneously.

Comparison with the CPU and OpenMP Versions

The CPU version performs the search serially using nested loops, so all 3×3 regions are checked one after another by a single thread. The OpenMP version keeps almost the same structure, but distributes regions across CPU threads using a reduction on `count`. Essentially, OpenMP parallelises the traversal while leaving the underlying branching behaviour largely unchanged.

The CUDA version restructures the algorithm around the GPU execution and memory hierarchy. Thousands of lightweight threads evaluate regions simultaneously, shared-memory tiling reduces repeated global-memory access, constant memory stores the glider patterns, and warp-level reduction reduces atomic contention.

For a small 16×16 image, the CPU and OpenMP versions can potentially outperform CUDA because the workload is too small to amortise CUDA memory-transfer, kernel-launch, and synchronisation overheads.

2.2.2 Histogram

Description of Approach

The CUDA Histogram implementation maps the input array onto a one-dimensional grid of GPU threads, where each thread processes one value from `numbers`. Each valid thread computes its global index using:

```
size_t i =  
    blockIdx.x * blockDim.x + threadIdx.x;
```

The kernel uses dynamically allocated shared memory to create one block-local histogram per thread block:

```
extern __shared__ int local_hist[];
```

Before traversal begins, threads cooperatively initialise `local_hist[]` to zero, after which `__syncthreads()` ensures the shared histogram is fully initialised before updates begin.

Each valid thread computes its bin and updates the shared-memory histogram:

```
int bin = numbers[i] / bin_width;  
  
atomicAdd(&local_hist[bin], 1);
```

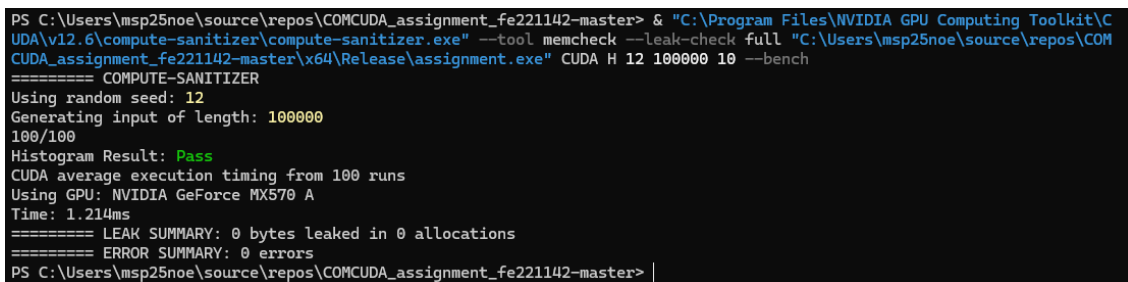
The atomic operation is required because several threads within the same block may attempt to update the same histogram bin simultaneously. After another `__syncthreads()`, the block-local histogram is merged into the global output histogram using additional atomic operations.

The kernel is launched using dynamically allocated shared memory:

```
histogram_kernel<<<
    num_blocks,
    threads_per_block,
    HISTOGRAM_LEN * sizeof(int)
>>>(...);
```

This gives each block enough shared memory to store one temporary histogram before the final merge into global memory.

The host function handles device allocation, memory transfers, output initialisation, kernel launch, result copy-back, and memory release.



```
PS C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master> & "C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.6\compute-sanitizer\compute-sanitizer.exe" --tool memcheck --leak-check full "C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master\x64\Release\assignment.exe" CUDA H 12 100000 10 --bench
===== COMPUTE-SANITIZER
Using random seed: 12
Generating input of length: 100000
100/100
Histogram Result: Pass
CUDA average execution timing from 100 runs
Using GPU: NVIDIA GeForce MX570 A
Time: 1.214ms
===== LEAK SUMMARY: 0 bytes leaked in 0 allocations
===== ERROR SUMMARY: 0 errors
PS C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master> |
```

Figure 2.2: Checking for Leaks and Errors in the CUDA Histogram Algorithm

Figure 2.2 shows the Compute Sanitizer leak-check result for the CUDA histogram implementation. The output reports no leaks and no detected errors.

Justification of Approach

The histogram traversal is naturally data-parallel because each input value can be processed independently. The main difficulty is that several threads may attempt to update the same histogram bin simultaneously.

For this reason, the implementation first builds a block-local histogram in shared memory before merging the partial results into the final global histogram. This reduces the number of direct global-memory atomic updates and follows the CUDA memory hierarchy more efficiently than updating global memory directly for every input value.

The use of `atomicAdd()` is still necessary because multiple values may map to the same bin. The first atomic protects updates inside the shared-memory histogram, while the second protects the final merge into global memory.

A block size of 256 threads was used because it is a multiple of the warp size 32, producing eight full warps per block.

Limitations of Approach

The main limitation is still atomic contention. Although shared memory reduces the number of global atomic updates, values that repeatedly fall into the same bin still force serialisation inside `local_hist[]`. Therefore, the performance depends partly on the distribution of the input values.

There is also a synchronisation cost. The kernel requires one barrier after clearing the shared histogram and another barrier before merging it into global memory. These barriers are necessary for correctness, but they reduce parallel freedom inside the block.

Finally, the shared-memory size depends on `HISTOGRAM_LEN`. This is appropriate for the given assignment range, but if the number of bins became very large, each block would require more shared memory, which could reduce occupancy.

Comparison with CPU and OpenMP Approach

The CPU version performs the histogram traversal serially and updates `output[bin]` directly, so no synchronisation is required.

The OpenMP version parallelises traversal using thread-local histograms followed by a merge. The CUDA version applies the same idea at block level: each block first builds a shared-memory histogram before merging into global memory using atomic operations.

Compared with OpenMP, CUDA exposes far more lightweight threads, making atomic contention and shared-memory locality more important. CUDA therefore benefits more clearly for larger inputs, where the available parallelism outweighs memory-transfer and kernel-launch overhead.

2.2.3 Emboss

Description of Approach

The CUDA Emboss implementation maps the output image onto a two-dimensional grid of GPU threads, where each thread computes one output pixel. The kernel is launched using two-dimensional blocks and grid dimensions computed from the output image size:

```
dim3 threadsPerBlock(  
    EMOSS_BLOCK_W,  
    EMOSS_BLOCK_H  
);  
  
dim3 blocksPerGrid(  
    (out_width + EMOSS_BLOCK_W - 1)
```

```

        / EMBOSS_BLOCK_W,
(out_height + EMBOSS_BLOCK_H - 1)
        / EMBOSS_BLOCK_H
);

```

Each block declares a shared-memory tile containing a block-local RGB image region:

```

__shared__ unsigned char
tile[EMBOSS_TILE_H][EMBOSS_TILE_W][3];

```

The tile includes halo pixels because the convolution uses a 3×3 stencil window. Threads cooperatively load the tile from global memory using a strided loop, after which `__syncthreads()` ensures all shared-memory values are available before convolution begins. The emboss mask is stored in constant memory because its coefficients are fixed and repeatedly accessed throughout execution:

```

__constant__ float d_emboss_kernel[9];

```

Each valid thread applies the emboss mask to its local 3×3 region in shared memory. RGB values are converted into greyscale, weighted by the stencil, accumulated into `pixel_sum`, normalised by adding 128, and clamped before being written to the output image:

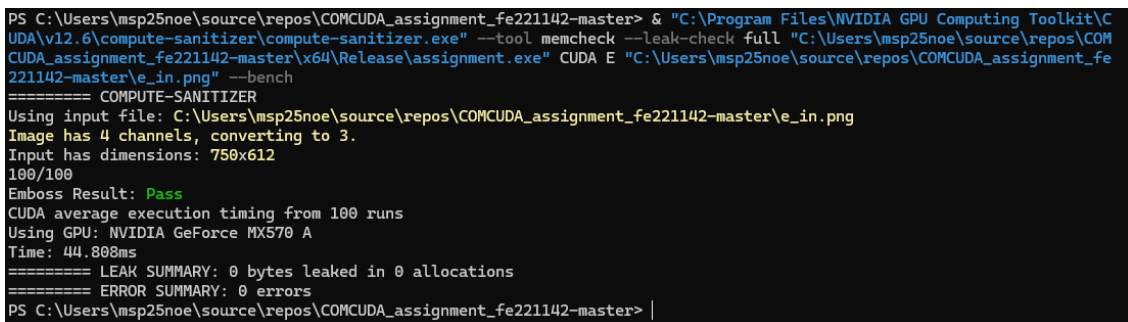
```

output[y * out_width + x] =
    clamp_to_uchar(pixel_sum);

```

Unlike Histogram, no atomic operations are required because each thread writes to a unique output pixel.

The host function handles device allocation, memory transfers, constant-memory initialisation, kernel launch, result copy-back, and memory release.



```

PS C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master> "C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.6\compute-sanitizer\compute-sanitizer.exe" --tool memcheck --leak-check full "C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master\emboss.exe" CUDA E "C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master\emboss_in.png" --bench
===== COMPUTE-SANITIZER
Using input file: C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master\emboss_in.png
Image has 4 channels, converting to 3.
Input has dimensions: 750x612
100/100
Emboss Result: Pass
CUDA average execution timing from 100 runs
Using GPU: NVIDIA GeForce MX570 A
Time: 44.808ms
===== LEAK SUMMARY: 0 bytes leaked in 0 allocations
===== ERROR SUMMARY: 0 errors
PS C:\Users\msp25noe\source\repos\COMCUDA_assignment_fe221142-master> |

```

Figure 2.3: Checking for Leaks and Errors in the CUDA Emboss Algorithm

Figure 2.3 shows the Compute Sanitizer result for the CUDA emboss implementation. The output reports no leaks and no detected errors.

Justification of Approach

A two-dimensional grid and two-dimensional thread blocks were used because the emboss operation is applied over a two-dimensional image. This gives a direct mapping between thread coordinates and output-pixel coordinates. The 16×16 block size gives 256 threads per block, corresponding to eight full warps.

Shared-memory tiling was used because neighbouring output pixels depend on overlapping 3×3 regions. Loading a larger tile together with halo pixels allows neighbouring threads to reuse RGB values locally instead of repeatedly fetching them from global memory.

The emboss mask was placed in constant memory because the coefficients are fixed and repeatedly accessed throughout convolution.

The algorithm also maps efficiently onto the SIMT execution model because threads perform almost identical stencil computations with very little branching or synchronisation overhead. Unlike Histogram, no atomic operations are required, and unlike Count Gliders, neighbouring threads rarely diverge in control flow.

Limitations of Approach

Although shared-memory tiling reduces repeated global-memory access, it introduces additional synchronisation overhead. Threads must first cooperatively load the shared tile and then wait at `__syncthreads()` before convolution can begin. This overhead becomes less worthwhile for smaller images where the computation itself is relatively limited.

The shared tile also requires additional memory beyond the exact block dimensions because halo pixels are needed for the 3×3 stencil window. Increasing tile size improves data reuse, but also increases shared-memory usage per block, which may reduce occupancy because fewer blocks can execute simultaneously on each SM.

Boundary handling also introduces some inefficiency near the image edges. Threads whose coordinates fall outside the valid output region either load zero-valued boundary pixels or terminate early, meaning some threads inside edge blocks perform less useful work than others.

The RGB image layout also requires three channel loads per pixel during convolution. This increases memory traffic compared with a single-channel greyscale image.

Comparison with CPU and OpenMP

The CPU, OpenMP, and CUDA implementations all perform the same stencil computation, greyscale conversion, normalisation, and clamping. The main difference lies in how the convolution work and memory accesses are organised across the hardware.

The CPU version computes output pixels serially using nested loops. The OpenMP version parallelises this traversal across CPU threads, but still relies mainly on the CPU cache hierarchy to exploit locality between overlapping 3×3 neighbourhoods.

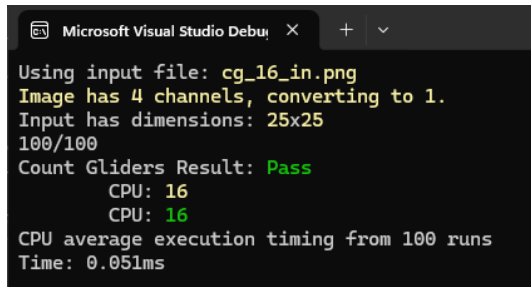
The CUDA version exposes substantially more fine-grained parallelism by assigning one GPU thread to each output pixel. Unlike OpenMP, the implementation also explicitly manages the GPU memory hierarchy through shared-memory tiling and constant memory. This is particularly effective for emboss because neighbouring threads repeatedly access overlapping RGB regions during convolution.

CUDA also introduces additional overhead through device allocation, memory transfer, kernel launch, and synchronisation. However, emboss maps efficiently onto the SIMT execution model because neighbouring threads perform almost identical stencil computations with very little branching or contention. The GPU implementation is therefore expected to benefit more clearly as image size increases.

Chapter 3

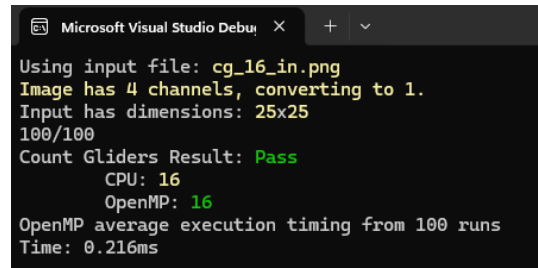
Benchmarking

3.1 Count Gliders



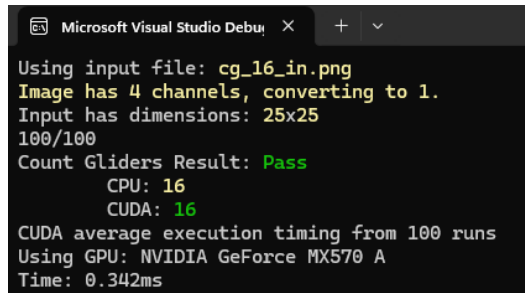
```
Microsoft Visual Studio Debug Console
Using input file: cg_16_in.png
Image has 4 channels, converting to 1.
Input has dimensions: 25x25
100/100
Count Gliders Result: Pass
CPU: 16
CPU: 16
CPU average execution timing from 100 runs
Time: 0.051ms
```

(a) Execution Time of Count Gliders Algorithm with the CPU Implementation



```
Microsoft Visual Studio Debug Console
Using input file: cg_16_in.png
Image has 4 channels, converting to 1.
Input has dimensions: 25x25
100/100
Count Gliders Result: Pass
CPU: 16
OpenMP: 16
OpenMP average execution timing from 100 runs
Time: 0.216ms
```

(b) Execution Time of Count Gliders Algorithm after OpenMP reformulation



```
Microsoft Visual Studio Debug Console
Using input file: cg_16_in.png
Image has 4 channels, converting to 1.
Input has dimensions: 25x25
100/100
Count Gliders Result: Pass
CPU: 16
CUDA: 16
CUDA average execution timing from 100 runs
Using GPU: NVIDIA GeForce MX570 A
Time: 0.342ms
```

(c) Execution Time of Count Gliders Algorithm after CUDA reformulation

Figure 3.1: Execution Time of Count Gliders

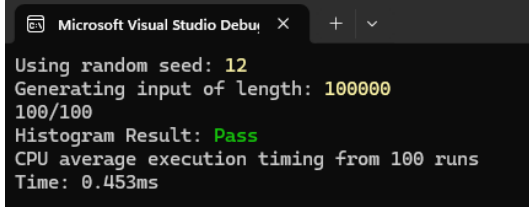
The Count Gliders benchmark favours the serial CPU implementation because the workload is extremely small. A 25×25 image produces only 529 valid 3×3 regions, so the matching work completes before OpenMP scheduling or CUDA launch overhead can become worthwhile.

OpenMP therefore adds thread-management overhead onto an already small search space, while CUDA spends proportionally more time on memory allocation, transfer, kernel launch, and synchronisation than on the matching work itself.

The algorithm is also relatively irregular. Different regions may terminate after different numbers

of comparisons depending on where mismatches occur during reflection and rotation checks. This creates uneven execution behaviour and branch divergence on the GPU, reducing parallel efficiency further for small inputs.

3.2 Histogram

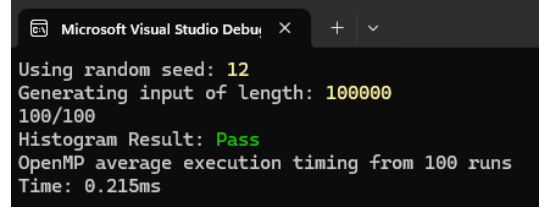


```

Microsoft Visual Studio Debug Console
Using random seed: 12
Generating input of length: 100000
100/100
Histogram Result: Pass
CPU average execution timing from 100 runs
Time: 0.453ms

```

(a) Execution Time of Histogram Algorithm with the CPU Implementation

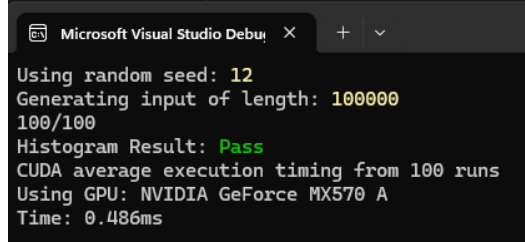


```

Microsoft Visual Studio Debug Console
Using random seed: 12
Generating input of length: 100000
100/100
Histogram Result: Pass
OpenMP average execution timing from 100 runs
Time: 0.215ms

```

(b) Execution Time of Histogram Algorithm after OpenMP reformulation



```

Microsoft Visual Studio Debug Console
Using random seed: 12
Generating input of length: 100000
100/100
Histogram Result: Pass
CUDA average execution timing from 100 runs
Using GPU: NVIDIA GeForce MX570 A
Time: 0.486ms

```

(c) Execution Time of Histogram Algorithm after CUDA reformulation

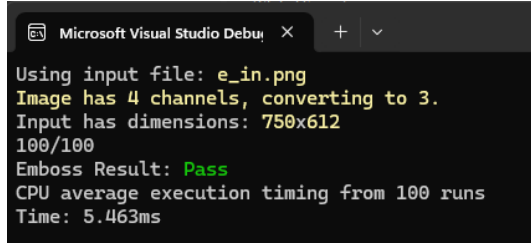
Figure 3.2: Execution Time of Histogram

The Histogram benchmark is dominated more by synchronisation than arithmetic work. OpenMP performs best because each thread builds its own local histogram before the final merge stage, avoiding contention during traversal.

The CUDA version reduces global-memory contention by first accumulating results inside shared memory, but threads within the same block may still update the same histogram bin simultaneously using `atomicAdd()`. These updates must therefore be serialised, reducing effective parallel throughput.

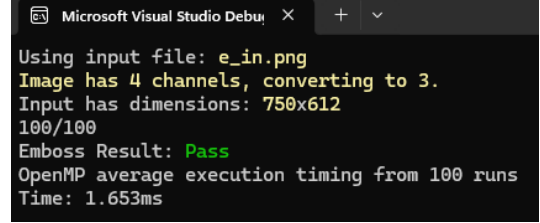
This benchmark reflects a common limitation with GPU parallelism, which is that it becomes less effective when many threads repeatedly compete for the same memory location.

3.3 Emboss



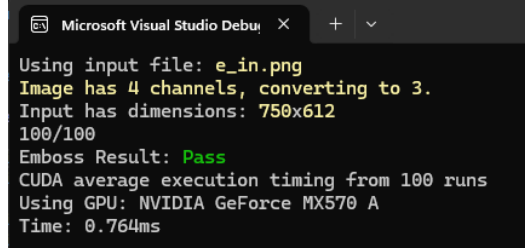
```
Microsoft Visual Studio Debug Console
Using input file: e_in.png
Image has 4 channels, converting to 3.
Input has dimensions: 750x612
100/100
Emboss Result: Pass
CPU average execution timing from 100 runs
Time: 5.463ms
```

(a) Execution Time of Emboss Algorithm with the CPU Implementation



```
Microsoft Visual Studio Debug Console
Using input file: e_in.png
Image has 4 channels, converting to 3.
Input has dimensions: 750x612
100/100
Emboss Result: Pass
OpenMP average execution timing from 100 runs
Time: 1.653ms
```

(b) Execution Time of Emboss Algorithm after OpenMP reformulation



```
Microsoft Visual Studio Debug Console
Using input file: e_in.png
Image has 4 channels, converting to 3.
Input has dimensions: 750x612
100/100
Emboss Result: Pass
CUDA average execution timing from 100 runs
Using GPU: NVIDIA GeForce MX570 A
Time: 0.764ms
```

(c) Execution Time of Emboss Algorithm after CUDA reformulation

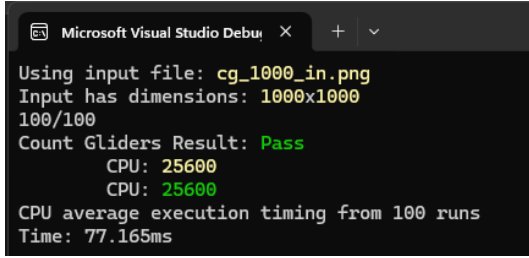
Figure 3.3: Execution Time of Emboss

Unlike the previous benchmarks, Emboss maps efficiently onto GPU execution. Each output pixel performs the same fixed 3×3 stencil operation independently of neighbouring pixels, producing regular control flow and independent output writes.

The workload is also substantially larger than in Count Gliders, so CUDA overhead becomes relatively small compared with the amount of computation being performed. The GPU can therefore keep many warps active simultaneously and sustain much higher throughput.

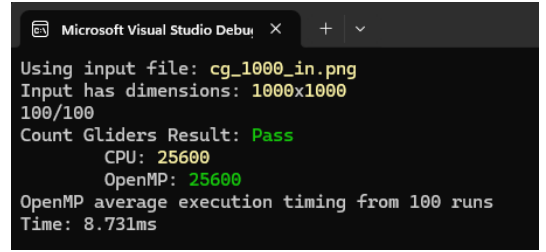
OpenMP improves significantly because output pixels are independent, but CUDA benefits further from the regular memory-access pattern and uniform thread behaviour. Unlike Histogram, the implementation requires no atomic operations, and unlike Count Gliders, there is very little warp divergence between neighbouring threads.

3.4 Count Gliders: Extra



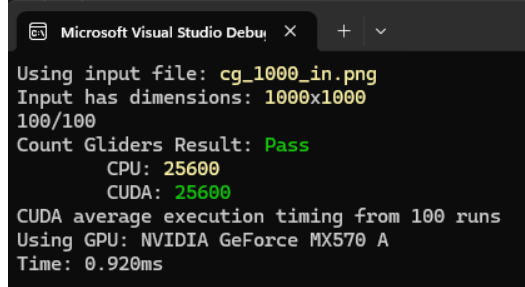
```
Microsoft Visual Studio Debug Console
Using input file: cg_1000_in.png
Input has dimensions: 1000x1000
100/100
Count Gliders Result: Pass
CPU: 25600
CPU: 25600
CPU average execution timing from 100 runs
Time: 77.165ms
```

(a) Execution Time of Count Glider 1000 Algorithm with the CPU Implementation



```
Microsoft Visual Studio Debug Console
Using input file: cg_1000_in.png
Input has dimensions: 1000x1000
100/100
Count Gliders Result: Pass
CPU: 25600
OpenMP: 25600
OpenMP average execution timing from 100 runs
Time: 8.731ms
```

(b) Execution Time of Count Glider 1000 Algorithm after OpenMP reformulation



```
Microsoft Visual Studio Debug Console
Using input file: cg_1000_in.png
Input has dimensions: 1000x1000
100/100
Count Gliders Result: Pass
CPU: 25600
CUDA: 25600
CUDA average execution timing from 100 runs
Using GPU: NVIDIA GeForce MX570 A
Time: 0.920ms
```

(c) Execution Time of Count Glider 1000 Algorithm after CUDA reformulation

Figure 3.4: Execution Time of Count Glider After Using A Different More Complex Image

For the larger Count Gliders benchmark, the performance trend changes substantially. The serial CPU implementation required 77.165 ms, OpenMP required 8.731 ms, and CUDA achieved the lowest execution time at 0.920 ms.

The earlier 25×25 benchmark was dominated by parallel overhead because the search space was too small. Here, the algorithm must process almost one million candidate regions:

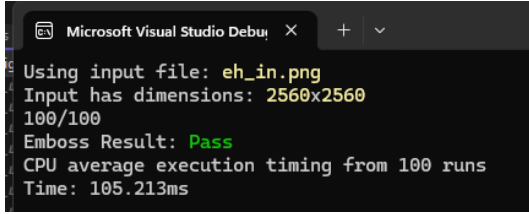
$$(1000 - 2)(1000 - 2)$$

At this scale, the matching work dominates execution time. OpenMP benefits because the traversal contains enough iterations to keep CPU threads occupied continuously, while CUDA overheads become relatively insignificant compared with the amount of parallel work available.

This therefore reflects a key idea, which is that GPU architectures achieve higher throughput when enough independent work exists to keep many warps active simultaneously.

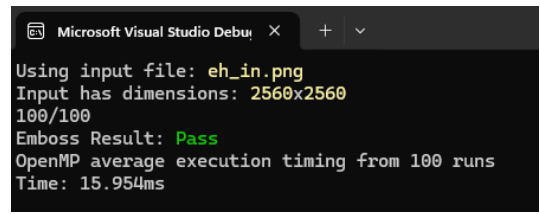
The larger benchmark image used for this experiment, `cg_1000_in.png`, is included in the submitted zip folder.

3.5 Emboss: Extra



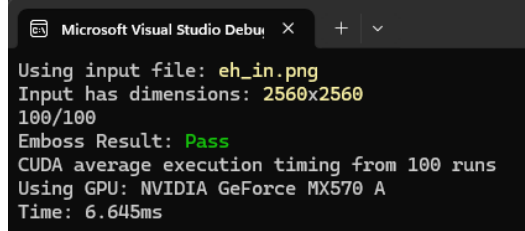
```
Microsoft Visual Studio Debug Console
Using input file: eh_in.png
Input has dimensions: 2560x2560
100/100
Emboss Result: Pass
CPU average execution timing from 100 runs
Time: 105.213ms
```

(a) Execution Time of Emboss Algorithm with the CPU Implementation



```
Microsoft Visual Studio Debug Console
Using input file: eh_in.png
Input has dimensions: 2560x2560
100/100
Emboss Result: Pass
OpenMP average execution timing from 100 runs
Time: 15.954ms
```

(b) Execution Time of Emboss Algorithm after OpenMP reformulation



```
Microsoft Visual Studio Debug Console
Using input file: eh_in.png
Input has dimensions: 2560x2560
100/100
Emboss Result: Pass
CUDA average execution timing from 100 runs
Using GPU: NVIDIA GeForce MX570 A
Time: 6.645ms
```

(c) Execution Time of Emboss Algorithm after CUDA reformulation

Figure 3.5: Execution Time of Emboss After Using a Different More Complex Input Image

For the larger Emboss benchmark, the same overall performance trend remains, but the gap between the implementations becomes larger. The serial CPU implementation required 105.213 ms, OpenMP required 15.954 ms, and CUDA achieved the lowest execution time at 6.645 ms.

Emboss scales well because each output pixel performs the same fixed stencil computation independently of neighbouring pixels. OpenMP therefore distributes the traversal evenly across CPU threads with very little synchronisation overhead, while CUDA benefits further because neighbouring threads execute almost identical instructions and write to independent output locations.

For the larger 2560×2560 image, the convolution work dominates execution time, making CUDA memory-transfer and kernel-launch overhead relatively insignificant.

The larger benchmark image used for this experiment, `eh_in.png`, is included in the submitted zip folder.

3.6 NSight Compute Profiling Results

We also used NSight Compute to profile the performance of all three CUDA algorithms and to take a better look at the bottlenecks within the code.

3.6.1 Count Gliders

#	Name	Start	Duration	TID
1	cuModuleGetLoadingMode	0.979534s	1.894 μ s	1348
2	cuDeviceGetLuid	0.979908s	345 ns	1348
3	cudaEventCreate	0.979954s	165.424 ms	1348
4	cudaEventCreate	1.14538s	762 ns	1348
5	cudaEventRecord	1.14538s	21.967 μ s	1348
6	cudaEventSynchronize	1.1454s	7.689 μ s	1348
7	cudaMemcpyToSymbol	1.14541s	1.127 ms	1348
11	cudaMemcpyToSymbol	1.14655s	6.691 μ s	1348
12	cudaMalloc	1.14656s	62.907 μ s	1348
13	cudaMalloc	1.14662s	3.312 μ s	1348
14	cudaMemcpy	1.14662s	7.349 μ s	1348
15	cudaMemset	1.14663s	12.359 μ s	1348
16	countGlidersKernel	1.14665s	91.111 μ s	1348
18	cudaDeviceSynchronize	1.14674s	95.230 μ s	1348
19	cudaMemcpy	1.14683s	26.131 μ s	1348
20	cudaFree	1.14686s	4.293 μ s	1348
21	cudaFree	1.14686s	92.768 μ s	1348
22	cudaEventRecord	1.14696s	9.076 μ s	1348
23	cudaEventSynchronize	1.14697s	3.365 μ s	1348
24	cudaEventDestroy	1.14697s	799 ns	1348
25	cudaEventDestroy	1.14697s	257 ns	1348
26	cuChxSynchronize	1.147s	757 ns	1348
27	cudaDeviceReset	1.14759s	12.795 ms	1348
28	cuLibraryUnload	1.1614s	68.516 μ s	1348
29	cuChxSynchronize	1.16457s	1.766 μ s	1348

Figure 3.6: Profiling Results of CUDA Implementation of Count Gliders

The profiling results show that `countGlidersKernel` required $91.111 \mu s$, while `cudaMemcpy` after execution required only $26.131 \mu s$. This shows that, for the larger workload, execution time is dominated more by the kernel's comparison work than by host-device transfer overhead.

The profile also shows relatively high synchronisation costs, with `cudaDeviceSynchronize` taking $95.230 \mu s$. This is consistent with what we know about synchronisation stalling the CPU until all GPU work completes, which makes it one of the more expensive runtime operations in CUDA programs.

3.6.2 Histogram

#	Name	Start	Duration	TID
1	cuModuleGetLoadingMode	0.983985s	1.900 μ s	26952
2	cuDeviceGetLuid	0.984054s	201 ns	26952
3	cudaEventCreate	0.984103s	69.557 ms	26952
4	cudaEventCreate	1.05366s	1.185 μ s	26952
5	cudaEventRecord	1.05366s	21.467 μ s	26952
6	cudaEventSynchronize	1.05369s	3.656 μ s	26952
7	cudaMalloc	1.05369s	86.702 μ s	26952
8	cudaMalloc	1.05378s	3.363 μ s	26952
9	cudaMemcpy	1.05378s	61.366 μ s	26952
10	cudaMemset	1.05384s	13.981 μ s	26952
10	cudaMemset	1.05509s	13.981 μ s	26952
11	histogram_kernel	1.05509s	1.190 ms	26952
16	cudaDeviceSynchronize	1.05505s	11.317 μ s	26952
17	cudaMemcpy	1.05506s	27.304 μ s	26952
18	cudaFree	1.05509s	4.866 μ s	26952
19	cudaFree	1.05509s	121.157 μ s	26952
20	cudaEventRecord	1.05521s	9.890 μ s	26952
21	cudaEventSynchronize	1.05522s	4.272 μ s	26952
22	cudaEventDestroy	1.05523s	952 ns	26952
23	cudaEventDestroy	1.05523s	250 ns	26952
18	cudaFree	1.05509s	4.866 μ s	26952
19	cudaFree	1.05509s	121.157 μ s	26952
20	cudaEventRecord	1.05521s	9.890 μ s	26952
21	cudaEventSynchronize	1.05522s	4.272 μ s	26952
22	cudaEventDestroy	1.05523s	952 ns	26952
23	cudaEventDestroy	1.05523s	250 ns	26952
24	cuChxSynchronize	1.05526s	900 ns	26952
25	cudaDeviceReset	1.05572s	13.106 ms	26952
26	cuLibraryUnload	1.06884s	68.746 μ s	26952
27	cuChxSynchronize	1.07231s	2.098 μ s	26952

Figure 3.7: Profiling Results of CUDA Implementation of Histogram

The profiling results show that `histogram_kernel` required $1.190 ms$, while the surrounding `cudaMemcpy` operations required only $61.366 \mu s$ and $27.304 \mu s$. This indicates that execution time is dominated by the kernel itself rather than memory-transfer overhead.

This behaviour aligns with the algorithm’s heavy use of `atomicAdd()`. Although the implementation reduces global-memory contention by first accumulating results inside block-local shared memory, threads within the same block may still attempt to update the same shared-memory histogram bin simultaneously. These conflicting atomic updates must therefore be serialised, which compromises effective parallel throughput despite the large number of active GPU threads.

3.6.3 Emboss

#	Name	Start	Duration	TID
1	cudaModuleGetLoadingMode	1.0063s	2.038 μ s	19160
2	cudaDeviceGetLuid	1.00652s	436 ns	19160
3	cudaEventCreate	1.00657s	128.390 ms	19160
4	cudaEventCreate	1.13496s	871 ns	19160
5	cudaEventRecord	1.13508s	18.467 μ s	19160
6	cudaEventSynchronize	1.1351s	3.347 μ s	19160
7	cudaMalloc	1.1351s	76.117 μ s	19160
8	cudaMalloc	1.13518s	2.900 μ s	19160
9	cudaMemcpy	1.13518s	140.543 μ s	19160
10	cudaMemcpyToSymbol	1.13532s	1.161 ms	19160

#	Name	Start	Duration	TID
8	cudaMalloc	1.13518s	2.900 μ s	19160
9	cudaMemcpy	1.13518s	140.543 μ s	19160
10	cudaMemcpyToSymbol	1.13532s	1.161 ms	19160
14	emboss_kernel	1.13648s	70.206 μ s	19160
16	cudaDeviceSynchronize	1.13655s	70.532 μ s	19160
17	cudaMemcpy	1.13662s	124.099 μ s	19160
18	cudaFree	1.13675s	5.125 μ s	19160
19	cudaFree	1.13675s	141.940 μ s	19160
20	cudaEventRecord	1.1369s	10.190 μ s	19160
21	cudaEventSynchronize	1.13691s	5.674 μ s	19160
22	cudaEventDestroy	1.13691s	1.033 μ s	19160
23	cudaEventDestroy	1.13692s	264 ns	19160
24	cudaDeviceSynchronize	1.13721s	2.050 μ s	19160
25	cudaDeviceReset	1.13789s	13.906 ms	19160
26	cuLibraryUnload	1.15181s	15.695 μ s	19160
27	cuCtxSynchronize	1.15837s	2.557 μ s	19160

#	Name	Start	Duration	TID
18	cudaFree	1.13675s	5.125 μ s	19160
19	cudaFree	1.13675s	141.940 μ s	19160
20	cudaEventRecord	1.1369s	10.190 μ s	19160
21	cudaEventSynchronize	1.13691s	5.674 μ s	19160
22	cudaEventDestroy	1.13691s	1.033 μ s	19160
23	cudaEventDestroy	1.13692s	264 ns	19160
24	cudaDeviceSynchronize	1.13721s	2.050 μ s	19160
25	cudaDeviceReset	1.13789s	13.906 ms	19160
26	cuLibraryUnload	1.15181s	15.695 μ s	19160
27	cuCtxSynchronize	1.15837s	2.557 μ s	19160

Figure 3.8: Profiling Results of CUDA Implementation of Emboss

Unlike Histogram, `emboss_kernel` required only 67.647μ s, while `cudaMemcpyToSymbol` required 1.161 ms. This indicates that the stencil computation itself maps efficiently onto the GPU and that a larger proportion of the measured runtime comes from setup and memory-transfer operations rather than the convolution work itself.

This behaviour obeys the regular structure of the emboss algorithm. Each thread performs the same fixed 3×3 stencil operation with relatively uniform control flow and predictable memory access.

Chapter 4

Conclusion

This report implemented and evaluated OpenMP and CUDA versions of the Count Gliders, Histogram, and Emboss algorithms through benchmarking and profiling. The results showed that OpenMP performed better for smaller or more irregular workloads with lower overhead, while CUDA performed best when the workload exposed large amounts of regular fine-grained parallelism.

The profiling results further showed how memory transfer, synchronisation, atomic contention, and execution regularity directly affected performance. In conclusion, this project demonstrated that successful parallel programming depends on matching algorithm structure to the architectural strengths of CPUs and GPUs.